CO2 - AT2 - DEBUGGING TASK

B.AMAR-192421397 ITA0612

Question 1: Perceptron Convergence Failure on Linearly Separable Data

1. Root Cause Analysis

According to Novikoff's Theorem, the perceptron learning algorithm is guaranteed to converge in a finite number of steps if the data is strictly linearly separable. Non-convergence implies system-level or implementation flaws:

- **Missing or Static Bias Term:** If the separating hyper-plane does not pass through the origin ($\mathbf{w}^T \mathbf{x} = 0$), omitting a bias parameter (or failing to append a constant dummy feature $x_0 = 1$ to input vectors) prevents the decision boundary from shifting.
- **Update Rule Flaws:** Common bugs include using soft predictions (probabilities) instead of step output ($y \in \{-1, +1\}$ or $\{0, 1\}$), sign inversion in the update rule ($\mathbf{w} \leftarrow \mathbf{w} - \eta (y - \hat{y})\mathbf{x}$ instead of $+ \eta$), or updating weights when the prediction is correct.
- **Dataset Preprocessing Errors:** Labels formatted as $\{0, 1\}$ mapped into a math formulation expecting $\{-1, +1\}$ breaks sign-based activation checks. • **Floating-Point Precision:** Extremely large magnitude differences between input features cause numerical instability and infinite oscillation around the decision boundary.

2. Systematic Debugging Strategies

1. **Isolate on Synthetic Data:** Test the pipeline on a 2D toy dataset with clear separation (e.g., $(1,1) \rightarrow +1$ and $(-1,-1) \rightarrow -1$).
2. **Track Weight Updates & Loss Logs:** Log $\Vert{}\mathbf{w}\Vert{}$ and the number of misclassified points per epoch. If misclassification counts cycle continuously without reaching 0, inspect the update condition.
3. **Inspect Augmented Inputs:** Confirm input vectors are transformed to $\mathbf{x}' = [1, x_1, x_2, \dots, x_d]^T$ so the bias updates concurrently with weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y_i - \hat{y}_i) \mathbf{x}_i$$

3. Optimization Techniques

- **Feature Normalization:** Apply Z-score standardization ($x' = \frac{x - \mu}{\sigma}$) to equalize scale across dimensions.
- **Margin Maxima (Pocket / Linear SVM):** Upgrade to the **Pocket Algorithm** to keep track of the best solution found so far, or transition to a soft-margin Support Vector Machine ($C\text{-SVM}$) for stability.

Question 2: Multilayer Perceptron (MLP) Slow Convergence & Local Minima

1. Root Cause Analysis

- **Vanishing/Exploding Gradients:** Deep architectures using Sigmoid or Tanh activations saturate in high/low input regimes, causing backpropagated gradients to decay exponentially ($\frac{\partial E}{\partial w} \approx 0$).
- **Saturating Initial Weights:** Initializing weights too large forces activations straight into saturation regions during epoch 1.
- **Saddle Points & Flat Regions:** High-dimensional non-convex optimization problems are dominated by saddle points rather than local minima, stopping standard Gradient Descent cold.

2. Step-by-Step Debugging

```
[Start Debugging MLP]
 │
 ├── 1. Monitor Gradient Norms layer-by-layer
 │   └── Small gradients at early layers? ──► Vanishing Gradient Problem  │
 ├── 2. Check Activation Outputs
 │   └── Mean close to 0 or 1 for Sigmoids? ──► Saturated Neurons  │
 └── 3. Perform Overfitting Test on 1 Batch
  └── Cannot hit 100% accuracy on 10 samples? ──► Code bug or rate  too
high/low
```

3. Optimization & Solutions

- **Activation Shift:** Replace Sigmoid/Tanh with **ReLU** or **Leaky ReLU** ($\max(\alpha x, x)$) in hidden layers to maintain constant non-saturating gradients.
- **He / Xavier Initialization:** Initialize weights based on layer fan-in/fan-out:

  $$\text{Xavier (Tanh): } W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right), \quad \text{He (ReLU): } W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

- **Advanced Optimizers:** Switch from Vanilla SGD to **Adam** or **RMSprop**, which maintain adaptive learning rates using first and second moment estimates (m_t, v_t).
- **Batch Normalization:** Normalize hidden layer inputs before activation:

  $$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Question 3: Genetic Algorithm (GA) Inconsistency & Suboptimal Results

1. Root Cause Analysis

- **Genetic Drift & Premature Convergence:** High selection pressure (e.g., strict tournament selection) causes a dominant non-optimal chromosome to duplicate quickly, dropping population diversity.
- **Flawed Fitness Landscape:** A deceptive or discontinuous fitness function that penalizes intermediate valid structures prevents traversal toward global optima.
- **Improper Encoding:** Binary representations of real-valued variables often introduce

Hamming cliffs (where $01111 \rightarrow 10000$ requires changing all bits, making small step changes unlikely via mutation).

2. Systematic Debugging Steps

- **Monitor Population Diversity Metric:** Compute average pairwise Hamming distance or phenotype variance across generations.
- **Track Best vs. Average Fitness:** Plot generational curves. If average fitness converges to best fitness too early, premature convergence has occurred.

3. Optimization Strategies

- **Adaptive Mutation Rates:** Dynamically increase mutation probability (P_m) when population diversity drops below a critical threshold.
- **Elitism Strategy:** Directly pass the top $k\%$ performing individuals to the next generation without modification to protect good solutions.
- **Selection Pressure Adjustments:** Replace proportional Roulette Wheel Selection with **Rank Selection** or lower the tournament size k in **Tournament Selection**. · **Real-Coded GA:** For continuous search spaces, switch from binary strings to real valued chromosomes with simulated binary crossover (SBX).

Question 4: Deep Neural Network + Genetic Programming Overfitting & High Cost

1. Root Cause Analysis

Combining Deep Learning (parameter estimation) with Genetic Programming (symbolic structure search) creates a massive search space. Overfitting occurs because GP grows overly complex symbolic trees (**bloat**) that fit noise, while DNN over-parameterization memorizes training samples.

2. Debugging Steps

1. **Identify Bloat:** Plot tree node count vs. fitness gains across generations. If tree size explodes with negligible fitness improvement, bloat is present.
2. **Evaluate Train-Validation Gap:** Measure loss curves on train vs. holdout sets for both GP tree structures and DNN sub-networks.

3. Optimization & Efficiency Techniques

Problem Domain	Technique	Implementation / Mech
GP Bloat	Parsimony Pressure	Add tree complexity pe into fitness: $F'(x) = F(x \text{Nodes}(x)$
DNN Overfitting	Regularization & Dropout	Apply L_2 weight dec \Vert{}W\Vert{}^2$) an neuron dropouts ($p=0
Compute Bottleneck	Surrogate Modeling	Substitute evaluation ca proxy approximations (for intermediate GP ge
Redundancy	Structural Pruning	Remove low-magnitude and dead-code branche post-training.

Question 5: Decision Tree Overfitting & Poor Generalization

1. Root Cause Analysis

· **Unbounded Tree Depth:** Splitting nodes until every leaf is pure ($Entropy = 0$) isolates individual training samples and captures sample-specific noise. · **Low Min-Samples Per Leaf:** Allowing leaves to contain very few samples (e.g., 1 or 2) leads to volatile decision boundaries.

· **Noisy / Irrelevant Features:** Tree algorithms split on non-informative features if they happen to yield small, spurious entropy reductions on the training set.

2. Debugging Steps

1. **Visualize Decision Boundaries:** Plot the decision surface for top features; look for highly fragmented "island-like" boundaries.
2. **Plot Depth vs. Error Curves:** Evaluate training and cross-validation performance across varying maximum depths (`max_depth`).

3. Optimization Techniques

- **Pre-Pruning (Early Stopping):** Limit `max_depth`, specify `min_samples_split`, and require `min_impurity_decrease`.
- **Post-Pruning (Cost-Complexity Pruning):** Minimize the objective function:

$$R_\alpha(T) = R(T) + \alpha \vert{}T\vert{}$$

where $R(T)$ is total leaf impurity, $\vert{}T\vert{}$ is node count, and α is the hyperparameter tuned via K-Fold Cross-Validation.

- **Feature Selection:** Remove low-information-gain attributes before building the tree.

Question 6: Convolutional Neural Network (CNN) Epoch Instability

1. Root Cause Analysis

- **Corrupted Data Augmentations:** Destructive transformations (e.g., random high intensity cropping, extreme rotation that obscures class-defining spatial cues) inject label noise during training.
- **Batch Normalization Issues:** Tiny batch sizes ($B < 8$) lead to unreliable batch statistics (μ_B, σ_B), causing high variance in feature maps. · **Unstable Learning Rate:** High learning rates cause the optimization trajectory to bounce off gradient walls in narrow loss canyons.

2. Debugging Steps

1. **Disable Augmentations:** Run training on static, clean data. If stability returns, isolate and fix aggressive transformation pipelines.
2. **Inspect Batch Size vs. Loss Variance:** Plot loss per step within an epoch. Spike patterns often correlate with tiny or heterogeneous batches.
3. **Check BN Phase Modes:** Verify proper switching between `model.train()` (computing batch statistics) and `model.eval()` (using running statistics).

3. Optimization Strategies

· **Layer Alternatives:** If batch sizes are small due to memory constraints, replace **Batch Normalization** with **Group Normalization** or **Layer Normalization**. · **Learning Rate Warmup & Schedulers:** Start with a low learning rate, linearly ramp up over N

epochs, and decay using a **Cosine Annealing** schedule:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right)\right)$$

- **Gradient Clipping:** Restrict exploding gradients by scaling norm values:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\text{threshold}}{\Vert{}\mathbf{g}\Vert{}}\right)$$

Question 7: Reinforcement Learning (RL) Policy Failure

1. Root Cause Analysis

- **Sparse or Deceptive Rewards:** If the reward is given only upon goal completion (e.g., $+1$ at the end of $1,000$ steps), the agent cannot credit intermediate useful actions.
- **Exploration-Exploitation Imbalance:** The agent exploits suboptimal local rewards early without sufficiently exploring the state-action space.
- **Partial Observability & Poor State Encoding:** The state representation lacks essential history (e.g., passing a single frame image without velocity/direction information).

2. Debugging Steps

1. **Log State-Value Estimates $V(s)$ or $Q(s,a)$:** Check for ungrounded value explosions or flatline zeros.
2. **Evaluate Random Policy Baseline:** Run a purely random agent to establish baseline step survival and action distribution metrics.
3. **Inspect Action Distributions:** Check if the policy collapses into repeating a single action continuously (policy degeneration).

3. Optimization Strategies

- **Reward Shaping:** Add auxiliary dense rewards to guide progress toward the goal without altering the optimal policy:

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

- **Advanced Exploration:** Move beyond simple ϵ-greedy to **Upper Confidence Bound (UCB)** or **Entropy Regularization** in policy gradient approaches:

$$\mathcal{L}_{total} = \mathcal{L}_{policy} - \beta \mathcal{H}(\pi(\cdot\vert{}s))$$

- **Frame Stacking & Normalization:** Stack k historical frames (e.g., 4 frames) to encode dynamics/velocity into state representations.

Question 8: K-Means Clustering Instability & Variable Assignments

1. Root Cause Analysis

· **Random Centroid Initial Sensitivity:** Standard K-means randomly selects initial centroids from data points, often leading to poor local optima.
· **Scale Sensitivity:** Features with larger numerical ranges dominate Euclidean distance computations ($d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_i - y_i)^2}$). · **Non-Spherical Geometry & Outliers:** K-means assumes spherical clusters of similar variance. Distance squared penalties make centroids pull heavily toward outliers.

2. Debugging Steps

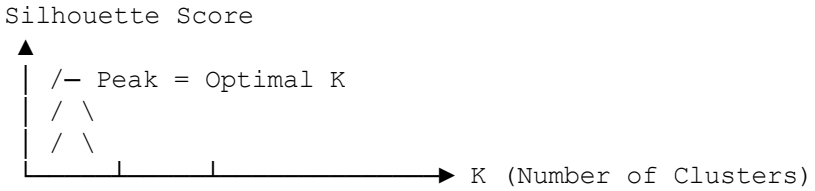
1. **Compute Inertia (WCSS) Distribution:** Run K-means 50 times with different seeds. High variance in final inertia confirms initial-state vulnerability. 2. **Evaluate Silhouette Plots:** Look for negative silhouette coefficients, which indicate samples placed in wrong clusters.

3. Optimization Strategies

· **K-Means++ Initialization:** Choose initial centroids probabilistically proportional to their squared distance from existing centroids:

$$P(x) = \frac{D(x)^2}{\sum D(x_i)^2}$$

· **Data Standardization:** Apply Robust Scaling or Z-Score Scaling prior to running distance computations.
· **Optimal Cluster Selection (K):** Use **Silhouette Analysis** combined with the **Elbow Method** (Inertia vs. K) to identify natural groupings.



Question 9: Support Vector Machine (SVM) Underperformance on Nonlinear Data

1. Root Cause Analysis

· **Linear Kernel Choice:** A standard linear kernel ($\mathbf{x}_i^T \mathbf{x}_j$) cannot separate nonlinearly separable decision boundaries.
· **Hyperparameter Mismatch (C and γ):**
  - Low C: Over-penalizes margins, causing severe underfitting.
  - Low γ (RBF Kernel): Treats variance too broadly, rendering decision boundaries nearly linear.
· **Unscaled Features:** Unscaled attributes dominate kernel calculations, distorting dual

coefficient estimates (α_i).

2. Debugging Steps

1. **Inspect Support Vector Counts:** If support vectors account for nearly 100% of training points, the model is overfitting or badly parameterized.
2. **Cross-Validation Grid Plots:** Heatmap validation accuracy across a 2D logarithmic grid of $C \in [10^{-3}, 10^3]$ and $\gamma \in [10^{-4}, 10^1]$.

3. Optimization Methods

- **Kernel Selection:** Switch to a **Radial Basis Function (RBF)** kernel:
  $$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \Vert{}\mathbf{x} - \mathbf{x}'\Vert{}^2\right)$$

- **Feature Scaling:** Standardize inputs using `StandardScaler` so every dimension operates with $\mu=0, \sigma=1$.
- **Hyperparameter Search:** Execute **GridSearchCV** or **Bayesian Optimization** to joint-tune C and γ.

Question 10: RNN Poor Long-Term Dependency Learning

1. Root Cause Analysis

Standard Recurrent Neural Networks (RNNs) suffer from **Vanishing and Exploding Gradients** during Backpropagation Through Time (BPTT). The repeated matrix multiplication of weight matrix W_{hh}^T over T timesteps leads to decay or growth:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \implies \text{If } \lambda_{max}(W_{hh}) < 1 \implies \text{decays to } 0 \text{ as } T \to \infty$$

2. Debugging Steps

1. **Track Timestep Gradients:** Log gradient norms at input timestep $t=1$ versus $t=T$. A drop to near 0 indicates vanishing gradients.
2. **Evaluate Memory Benchmarks:** Train the model on an synthetic task (e.g., retrieving a specific token from 100 steps prior).

3. Proposed Solutions

```
Standard RNN: h_t = tanh(W_h * h_{t-1} + W_x * x_t) [Single product pathway]

LSTM Gated Cell:
 ├── Forget Gate: f_t = σ(W_f · [h_{t-1}, x_t] + b_f)
 ├── Input Gate: i_t = σ(W_i · [h_{t-1}, x_t] + b_i)
 ├── Cell Update: C_t = f_t * C_{t-1} + i_t * C~_t <-- Constant Error Carousel
 └── Output Gate: o_t = σ(W_o · [h_{t-1}, x_t] + b_o)
```

- **Architecture Upgrade (LSTM / GRU):** Replace vanilla units with **Long Short Term Memory (LSTM)** or **Gated Recurrent Units (GRU)**. LSTMs maintain an additive Cell State (C_t) that acts as a "constant error carousel" to prevent gradient decay.
- **Attention Mechanisms:** Integrate **Self-Attention** or **Transformer Layers**. Attention forms direct paths between distant tokens (T_{now} and T_{past}), bypassing sequential recurrence bottlenecks entirely:

  $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Gradient Clipping:** Apply norm-based gradient clipping to handle exploding gradients during training.